

Кейс 5: «Антискам тренажер»

На классифайдах пользователи часто общаются напрямую: обсуждают товар, договариваются о встрече, уточняют доставку, выбирают способ оплаты. В таких сценариях могут возникать риски: собеседник просит перейти в сторонний мессенджер, отправляет ссылку на «безопасную оплату», просит назвать код из SMS или настаивает на предоплате вне платформы.

Платформа может технически бороться с мошенничеством: блокировать подозрительные ссылки, отслеживать жалобы, анализировать поведение пользователей и предупреждать о рисках. Но часть безопасности все равно зависит от решений самого пользователя. Важно, чтобы человек смог распознать тревожные признаки до того, как совершит рискованное действие.

Обычные текстовые инструкции помогают не всегда. Поэтому эффективнее работает интерактивный тренажер: пользователь проходит симуляцию диалога, выбирает действия в спорных моментах, видит последствия решения и получает объяснение, какие признаки риска были в сценарии.

В рамках кейса вам нужно создать обучающее веб-приложение в формате интерактивного тренажера безопасных сделок.

Бизнес-задача

Создайте MVP веб-приложения, которое:

Учит разные группы пользователей распознавать риски в сделках

Пользователь должен пройти ситуации, похожие на реальные, и тренироваться принимать безопасные решения.

Показывает последствия выбора

Тренажер должен не просто фиксировать правильный или неправильный ответ, а объяснять, почему выбранное действие помогает или, наоборот, повышает риск.

Формирует безопасное поведение внутри платформы

После прохождения обучения пользователь должен лучше понимать, на какие признаки обращать внимание и как действовать в подобной реальной ситуации.

Мотивирует продолжать обучение

Пользователь должен видеть результат прохождения и иметь повод вернуться к тренажеру, чтобы улучшить свой уровень безопасности.

Главная цель — создать обучающий продукт, который помогает пользователю перенести опыт тренировки в реальные сделки.

Что нужно реализовать

Пользователь выбирает роль покупателя или продавца, проходит один или несколько сценариев общения и получает оценку своих решений. Как выглядит интерфейс тренажера и как с ним взаимодействует пользователь, как тренажер интегрирован в платформу и в какой момент присутствия пользователя на сайте мы предлагаем ему пройти обучение вы определяете самостоятельно.

Определите ключевые ситуации, над которыми вам предстоит работать

Продумайте несколько тестовых рисков как на стороне покупателя, так и продавца. С какими видами мошенничества может столкнуться каждый из типов пользователей?

Разработайте пошаговый пользовательский сценарий

Пользователь выбирает роль: покупатель или продавец, проходит обучение и принимает решения в спорных моментах сделки. Ситуации должны быть достаточно реалистичными, чтобы пользователь узнавал в них возможные случаи из работы классифайда, но не должны превращаться в набор очевидных вопросов.

Реализуйте механизмы оценки и обратной связи

Тренажер должен давать пользователю мотивирующую оценку прохождения. Формат оценки команда выбирает самостоятельно.

Пользователь должен понять, какие решения помогали сохранить безопасность и что стоит запомнить перед реальной сделкой. Объяснение должно обучать, а не просто сообщать «правильно» или «неправильно».

Визуализируйте прогресс пользователя

Пользователь должен видеть свой прогресс в тренажере: какие тренировки он уже проходил, как менялся результат и куда можно перейти дальше. Хорошее решение помогает пользователю увидеть, что он лучше распознает риски после повторного прохождения.

Команда сама определяет способ отображения прогресса.

Сформулируйте и опишите документацию

Опишите какой пользовательский путь вы проектируете и как устроена предложенная вами система обучения.

Продумайте и предложите дополнительные функции

Они усилят решение, если работают на бизнес-задачу. Важно: дополнительные функции не должны заменять основную механику.

Важно

Вам нужно создать не чат с вопросами, а полноценный обучающий продуктовый сценарий. Важно, чтобы пользователь понимал, чему его учит тренажер и как это связано с безопасным поведением в настоящей сделке.

Удачное решение сочетает реалистичные ситуации для отработки, мотивирующую систему оценки и полезную информацию.

Требования по стеку

Frontend

- React 18+
- Typescript

Разрешено использование любых внешних библиотек, в том числе:

- Дизайн-система/UI-kit (Material-UI, Ant Design)
- Стейт-менеджмент (Redux, MobX, Effector)
- Линтер (ESLint)
- Prettier
- Система сборки (Webpack, Vite)
- Библиотека для работы с асинхронными HTTP-запросами (React Query, Axios)

Backend

- Язык программирования – Golang
- База данных — выбрать самостоятельно, к выбору приложить обоснование, почему выбрана та или иная БД.
- Деплой — приложение поднимается через Docker Compose
- Описать конфигурацию линтера (.golangci.yaml в корне проекта)

Оформление решения

- | | | |
|--|---|-------------------------|
| • Общий репозиторий с frontend + backend | • Приложение поднимается через docker-compose | • Описанный README файл |
|--|---|-------------------------|

Использование ИИ

- Вы можете пользоваться нейросетями, но делайте это прозрачно. Укажите дополнительно – на каких этапах и каким образом вы ими пользовались
- Если сможете интегрировать ИИ в ваш MVP — это станет плюсом при оценке вашего решения

Как жюри будет оценивать ваш результат

По каждому пункту можно получить от 0 до 5 баллов.

- **Обязательная бизнес-логика проекта**

Отличный результат: основной сценарий кейса работает полностью - пользователь может пройти путь от начала до конца, все ключевые состояния предусмотрены, логика соответствует бизнес-задаче.

- **Реализованный доп. функционал**

Отличный результат: дополнительные функции усиливают бизнес-задачу, органично выглядят в сценарии и не мешают обязательному MVP.

- **Наличие README**

Отличный результат: README позволяет запустить и проверить проект без устных пояснений команды; описаны запуск, зависимости, сценарии, API, ограничения MVP и ключевые решения.

- **Наличие тестов**

Отличный результат: покрытие юнит-тестами самых важных участков кода.

- **Наличие линтера**

Отличный результат: наличие на бэкенде и фронтенде линтеров с правилами и их обоснованием.

- **Дизайн и архитектура приложения**

Отличный результат: интерфейс понятный и аккуратный, архитектура проекта логичная, части приложения разделены по зонам ответственности.

- **Продуктовые решения, позволяющие достичь бизнес цели**

Отличный результат: понятное обоснование продуктовых решений, которыми достигается бизнес-цель.

- **Чистота и качество кода, консистентность стиля**

Отличный результат: код читаемый, структурированный, с понятными названиями, без мертвого кода и костылей. Настроены инструменты автоматической проверки качества.

- **Доступность в интернете**

Отличный результат: проект развернут и доступен по ссылке, жюри может проверить основной сценарий без локального запуска.